

LAB ACTIVITY (Lab-06)

1. Bank Account System

- Create a base class **Account** with attributes: **accountNumber**, **balance**.
- Add a constructor to initialize values.
- Create subclasses **SavingsAccount** and **CurrentAccount** that inherit **Account**.
- Implement **method overloading** for **deposit()** (e.g., deposit with **int** or **double**).
- Implement **method overriding** for **withdraw()** in subclasses with different rules (e.g., savings has minimum balance, current has overdraft).

```
class Account {
    String accNo;
    double balance;

    // Constructor
    Account(String accNo, double balance) {
        this.accNo = accNo;
        this.balance = balance;
    }

    // Deposit with no amount (just showing account details)
    void deposit() {
        System.out.println("Deposit operation:");
        System.out.println("Account No: " + accNo + ", Balance: " + balance);
    }

    // Overloaded deposit methods
    void deposit(int amount) {
        balance += amount;
        System.out.println("Deposited: " + amount +
            " | Account No: " + accNo +
            " | Balance: " + balance);
    }

    void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: " + amount +
            " | Account No: " + accNo +
            " | Balance: " + balance);
    }

    // Withdraw (base version)
    void withdraw(double amount) {
        System.out.println("Withdraw request: " + amount);
    }
}

// Savings Account
class SavingsAccount extends Account {
    public static final double MIN_BALANCE = 5000.00;

    SavingsAccount(String accNo, double balance) {
```

```

    super(accNo, balance);
}

@Override
void withdraw(double amount) {
    if (balance - amount >= MIN_BALANCE) {
        balance -= amount;
        System.out.println("Withdrawal of " + amount + " successful. Updated Balance: " + balance);
    } else {
        System.out.println("Withdrawal denied! Minimum balance of " + MIN_BALANCE + " must be
maintained.");
    }
}
}

// Current Account
class CurrentAccount extends Account {
    public static final double LOCK_AMOUNT = 1000.00;

    CurrentAccount(String accNo, double balance) {
        super(accNo, balance);
    }

    @Override
    void withdraw(double amount) {
        if (balance - amount >= LOCK_AMOUNT) {
            balance -= amount;
            System.out.println("Withdrawal of " + amount + " successful. Updated Balance: " + balance);
        } else {
            System.out.println("Withdrawal denied! Lock amount of " + LOCK_AMOUNT + " must remain in
account.");
        }
    }
}

// Main Class
public class MainBank {
    public static void main(String[] args) {

        // Base Account
        Account a1 = new Account("40004007", 2000.50);
        a1.deposit();
        a1.deposit(1000);
        a1.deposit(275.50);
        a1.withdraw(10000);

        System.out.println("-----");

        // Savings Account
        SavingsAccount s1 = new SavingsAccount("40004008", 7000.00);
        s1.deposit();
        s1.deposit(10000);
        s1.deposit(2755.50);
        s1.withdraw(1200);
        s1.withdraw(6000);

        System.out.println("-----");

        // Current Account

```

```

CurrentAccount c1 = new CurrentAccount("40004009", 5000.00);
c1.deposit();
c1.deposit(1000);
c1.deposit(275.50);
c1.withdraw(1500);
c1.withdraw(4500);
}
}

```

```

PS D:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)> cd "d:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)\
" ; if ($?) { java MainBank }
Deposit operation:
Account No: 40004007, Balance: 2000.5
Deposited: 1000 | Account No: 40004007 | Balance: 3000.5
Deposited: 275.5 | Account No: 40004007 | Balance: 3276.0
Withdraw request: 10000.0
-----
Deposit operation:
Account No: 40004008, Balance: 7000.0
Deposited: 10000 | Account No: 40004008 | Balance: 17000.0
Deposited: 2755.5 | Account No: 40004008 | Balance: 19755.5
Withdrawal of 1200.0 successful. Updated Balance: 18555.5
Withdrawal of 6000.0 successful. Updated Balance: 12555.5
-----
Deposit operation:
Account No: 40004009, Balance: 5000.0
Deposited: 1000 | Account No: 40004009 | Balance: 6000.0
Deposited: 275.5 | Account No: 40004009 | Balance: 6275.5
Withdrawal of 1500.0 successful. Updated Balance: 4775.5
Withdrawal denied! Lock amount of 1000.0 must remain in account.

```

2. Vehicle Hierarchy

- Create a base class **Vehicle** with attributes **speed** and **fuel**.
- Add a constructor.
- Create subclasses: **Car**, **Bike**, **Truck**.
- Use **method overloading** for **accelerate()** (accelerate with or without specifying gear).
- Use **method overriding** to provide different mileage calculation logic in each subclass.

```

// Base Class
class Vehicle {
    protected int speed; // in km/h
    protected double fuel; // in liters

    // Constructor
    public Vehicle(int speed, double fuel) {
        this.speed = speed;
        this.fuel = fuel;
    }

    // Method overloading: accelerate
    public void accelerate() {
        speed += 10;
        System.out.println("Vehicle accelerated by 10 km/h. Current speed: " + speed);
    }

    public void accelerate(int gear) {
        speed += gear * 10;
        System.out.println("Vehicle accelerated with gear " + gear + ". Current speed: " + speed);
    }
}

```

```

    }

    // Method to be overridden
    public double mileage() {
        System.out.println("Generic vehicle mileage calculation.");
        return 0.0;
    }

    public void display() {
        System.out.println("Speed: " + speed + " km/h, Fuel: " + fuel + " liters");
    }
}

// Car subclass
class Car extends Vehicle {
    public Car(int speed, double fuel) {
        super(speed, fuel);
    }

    @Override
    public double mileage() {
        double mileage = fuel * 15; // assume 15 km/l
        System.out.println("Car mileage: " + mileage + " km");
        return mileage;
    }
}

// Bike subclass
class Bike extends Vehicle {
    public Bike(int speed, double fuel) {
        super(speed, fuel);
    }

    @Override
    public double mileage() {
        double mileage = fuel * 40; // assume 40 km/l
        System.out.println("Bike mileage: " + mileage + " km");
        return mileage;
    }
}

// Truck subclass
class Truck extends Vehicle {
    public Truck(int speed, double fuel) {
        super(speed, fuel);
    }

    @Override
    public double mileage() {
        double mileage = fuel * 8; // assume 8 km/l
        System.out.println("Truck mileage: " + mileage + " km");
        return mileage;
    }
}

// Main class
public class MainVehicle {
    public static void main(String[] args) {
        // Car

```

```

Car car = new Car(60, 20);
car.display();
car.accelerate();
car.accelerate(2);
car.mileage();

System.out.println("-----");

// Bike
Bike bike = new Bike(40, 5);
bike.display();
bike.accelerate();
bike.accelerate(3);
bike.mileage();

System.out.println("-----");

// Truck
Truck truck = new Truck(50, 50);
truck.display();
truck.accelerate();
truck.accelerate(4);
truck.mileage();
}
}

```

```

PS D:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)> cd "d:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)\
ava } ; if ($?) { java MainVehicle }
Speed: 60 km/h, Fuel: 20.0 liters
Vehicle accelerated by 10 km/h. Current speed: 70
Vehicle accelerated with gear 2. Current speed: 90
Car mileage: 300.0 km
-----
Speed: 40 km/h, Fuel: 5.0 liters
Vehicle accelerated by 10 km/h. Current speed: 50
Vehicle accelerated with gear 3. Current speed: 80
Bike mileage: 200.0 km
-----
Speed: 50 km/h, Fuel: 50.0 liters
Vehicle accelerated by 10 km/h. Current speed: 60
Vehicle accelerated with gear 4. Current speed: 100
Truck mileage: 400.0 km
PS D:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)> 

```

3. Shape Area Calculator

- Create a base class **Shape**.
- Create subclasses: **Circle**, **Rectangle**, **Triangle**.
- Add constructors for each subclass.
- Implement **method overloading** in **Rectangle** (area with length & breadth OR with side only for square).
- Implement **method overriding** of **calculateArea()** for each shape.

```

// Base Class
abstract class Shape {
    // Abstract method to be overridden
    public abstract double calculateArea();
}

```

```
}

// Circle subclass
class Circle extends Shape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        double area = Math.PI * radius * radius;
        System.out.println("Circle area: " + area);
        return area;
    }
}

// Rectangle subclass
class Rectangle extends Shape {
    private double length;
    private double breadth;

    // Constructor for rectangle
    public Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;
    }

    // Constructor for square
    public Rectangle(double side) {
        this.length = side;
        this.breadth = side;
    }

    // Method overloading for area
    public double area(double length, double breadth) {
        return length * breadth;
    }

    public double area(double side) {
        return side * side;
    }

    @Override
    public double calculateArea() {
        double area = length * breadth;
        System.out.println("Rectangle area: " + area);
        return area;
    }
}

// Triangle subclass
class Triangle extends Shape {
    private double base;
    private double height;

    public Triangle(double base, double height) {
        this.base = base;
    }
}
```

```

        this.height = height;
    }

    @Override
    public double calculateArea() {
        double area = 0.5 * base * height;
        System.out.println("Triangle area: " + area);
        return area;
    }
}

// Main class
public class MainShapeArea {
    public static void main(String[] args) {
        // Circle
        Circle circle = new Circle(7);
        circle.calculateArea();

        System.out.println("-----");

        // Rectangle
        Rectangle rect = new Rectangle(10, 5);
        rect.calculateArea();
        System.out.println("Overloaded method (Rectangle 8x4): " + rect.area(8, 4));
        System.out.println("Overloaded method (Square 6): " + rect.area(6));

        System.out.println("-----");

        // Triangle
        Triangle tri = new Triangle(10, 6);
        tri.calculateArea();
    }
}

PS D:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)> cd "d:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)
.java" ; if ($?) { java MainShapeArea }
Circle area: 153.93804002589985
-----
Rectangle area: 50.0
Overloaded method (Rectangle 8x4): 32.0
Overloaded method (Square 6): 36.0
-----
Triangle area: 30.0
PS D:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)>

```

4. Employee Payroll System

- Create a base class **Employee** with attributes: **name**, **salary**.
- Add constructors (default + parameterized).
- Create subclasses: **Manager**, **Developer**, **Intern**.
- Implement **method overloading** for **calculateBonus()** (bonus with percentage OR fixed amount).
- Implement **method overriding** in subclasses to calculate bonus differently based on role.

```

// Base class
class Employee {
    protected String name;
    protected double salary;
}

```

```

// Default constructor
public Employee() {
    this.name = "Unknown";
    this.salary = 0.0;
}

// Parameterized constructor
public Employee(String name, double salary) {
    this.name = name;
    this.salary = salary;
}

// Method overloading: calculateBonus
public double calculateBonus(double percentage) {
    double bonus = (salary * percentage) / 100;
    System.out.println(name + " bonus (" + percentage + "%): " + bonus);
    return bonus;
}

public double calculateBonus(int fixedAmount) {
    System.out.println(name + " fixed bonus: " + fixedAmount);
    return fixedAmount;
}

// Method to override in subclasses
public double calculateBonus() {
    System.out.println("Generic employee bonus calculation.");
    return 0.0;
}

public void display() {
    System.out.println("Employee: " + name + ", Salary: " + salary);
}
}

// Manager subclass
class Manager extends Employee {
    public Manager(String name, double salary) {
        super(name, salary);
    }

    @Override
    public double calculateBonus() {
        double bonus = salary * 0.20; // Managers get 20% bonus
        System.out.println("Manager " + name + " bonus: " + bonus);
        return bonus;
    }
}

// Developer subclass
class Developer extends Employee {
    public Developer(String name, double salary) {
        super(name, salary);
    }

    @Override
    public double calculateBonus() {
        double bonus = salary * 0.15; // Developers get 15% bonus
    }
}

```

```

        System.out.println("Developer " + name + " bonus: " + bonus);
        return bonus;
    }
}

// Intern subclass
class Intern extends Employee {
    public Intern(String name, double salary) {
        super(name, salary);
    }

    @Override
    public double calculateBonus() {
        double bonus = 1000; // Interns get a fixed stipend bonus
        System.out.println("Intern " + name + " bonus: " + bonus);
        return bonus;
    }
}

// Main class
public class MainEmployee {
    public static void main(String[] args) {
        // Manager
        Manager m1 = new Manager("Alice", 80000);
        m1.display();
        m1.calculateBonus();
        m1.calculateBonus(10.0); // Overloaded
        m1.calculateBonus(5000); // Overloaded

        System.out.println("-----");

        // Developer
        Developer d1 = new Developer("Bob", 60000);
        d1.display();
        d1.calculateBonus();
        d1.calculateBonus(12.5);
        d1.calculateBonus(3000);

        System.out.println("-----");

        // Intern
        Intern i1 = new Intern("Charlie", 20000);
        i1.display();
        i1.calculateBonus();
        i1.calculateBonus(5.0);
        i1.calculateBonus(1000);
    }
}

```

```
PS D:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)> cd "d:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)"
java } ; if ($?) { java MainEmployee }
Employee: Alice, Salary: 80000.0
Manager Alice bonus: 16000.0
Alice bonus (10.0%): 8000.0
Alice fixed bonus: 5000
-----
Employee: Bob, Salary: 60000.0
Developer Bob bonus: 9000.0
Bob bonus (12.5%): 7500.0
Bob fixed bonus: 3000
-----
Employee: Charlie, Salary: 20000.0
Intern Charlie bonus: 1000.0
Charlie bonus (5.0%): 1000.0
Charlie fixed bonus: 1000
```

5. Library Management System

- Create a base class **Book** with attributes: **title**, **author**, **price**.
- Add constructors.
- Create subclasses: **EBook** and **PrintedBook**.
- Use **method overloading** for **applyDiscount()** (apply discount by percentage or fixed amount).
- Use **method overriding** to calculate **getFinalPrice()** differently for **EBook** (download fee) and **PrintedBook** (printing/shipping cost).

```
// Base class
class Book {
    protected String title;
    protected String author;
    protected double price;

    // Default constructor
    public Book() {
        this.title = "Unknown";
        this.author = "Unknown";
        this.price = 0.0;
    }

    // Parameterized constructor
    public Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price;
    }

    // Method overloading: applyDiscount
    public void applyDiscount(double percentage) {
        double discount = (price * percentage) / 100;
        price -= discount;
        System.out.println("Applied " + percentage + "% discount. New price: " + price);
    }

    public void applyDiscount(int amount) {
        price -= amount;
        if (price < 0) price = 0; // Prevent negative price
        System.out.println("Applied fixed discount of " + amount + ". New price: " + price);
    }
}
```

```

// Method to override
public double getFinalPrice() {
    System.out.println("Generic book final price calculation.");
    return price;
}

public void display() {
    System.out.println("Title: " + title + " | Author: " + author + " | Price: " + price);
}
}

// EBook subclass
class EBook extends Book {
    private double downloadFee = 50.0; // Extra cost for download

    public EBook(String title, String author, double price) {
        super(title, author, price);
    }

    @Override
    public double getFinalPrice() {
        double finalPrice = price + downloadFee;
        System.out.println("EBook Final Price (including download fee): " + finalPrice);
        return finalPrice;
    }
}

// PrintedBook subclass
class PrintedBook extends Book {
    private double shippingCost = 100.0; // Extra cost for shipping/printing

    public PrintedBook(String title, String author, double price) {
        super(title, author, price);
    }

    @Override
    public double getFinalPrice() {
        double finalPrice = price + shippingCost;
        System.out.println("Printed Book Final Price (including shipping): " + finalPrice);
        return finalPrice;
    }
}

// Main class
public class MainLibrary {
    public static void main(String[] args) {
        // EBook
        EBook ebook = new EBook("Digital Java", "Alice", 500);
        ebook.display();
        ebook.applyDiscount(10); // 10% discount
        ebook.applyDiscount(50); // Fixed discount
        ebook.getFinalPrice();

        System.out.println("-----");

        // PrintedBook
        PrintedBook pbook = new PrintedBook("Learn OOP", "Bob", 800);
        pbook.display();
        pbook.applyDiscount(20); // 2

```

```
}  
}  
  
PS D:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)> cd "d:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)\";  
java } ; if ($?) { java MainLibrary }  
Title: Digital Java | Author: Alice | Price: 500.0  
Applied fixed discount of 10. New price: 490.0  
Applied fixed discount of 50. New price: 440.0  
EBook Final Price (including download fee): 490.0  
-----  
Title: Learn OOP | Author: Bob | Price: 800.0  
Applied fixed discount of 20. New price: 780.0  
PS D:\SEMESTER I\LAB WORK\Java Programming\lab6(1309)> |
```